

The background of the image features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

D O P 3 1 3 - R

Multi-account and multi-Region deployments at scale

James Bland, PhD

Principal Solutions Architect
AWS

Welly Siau

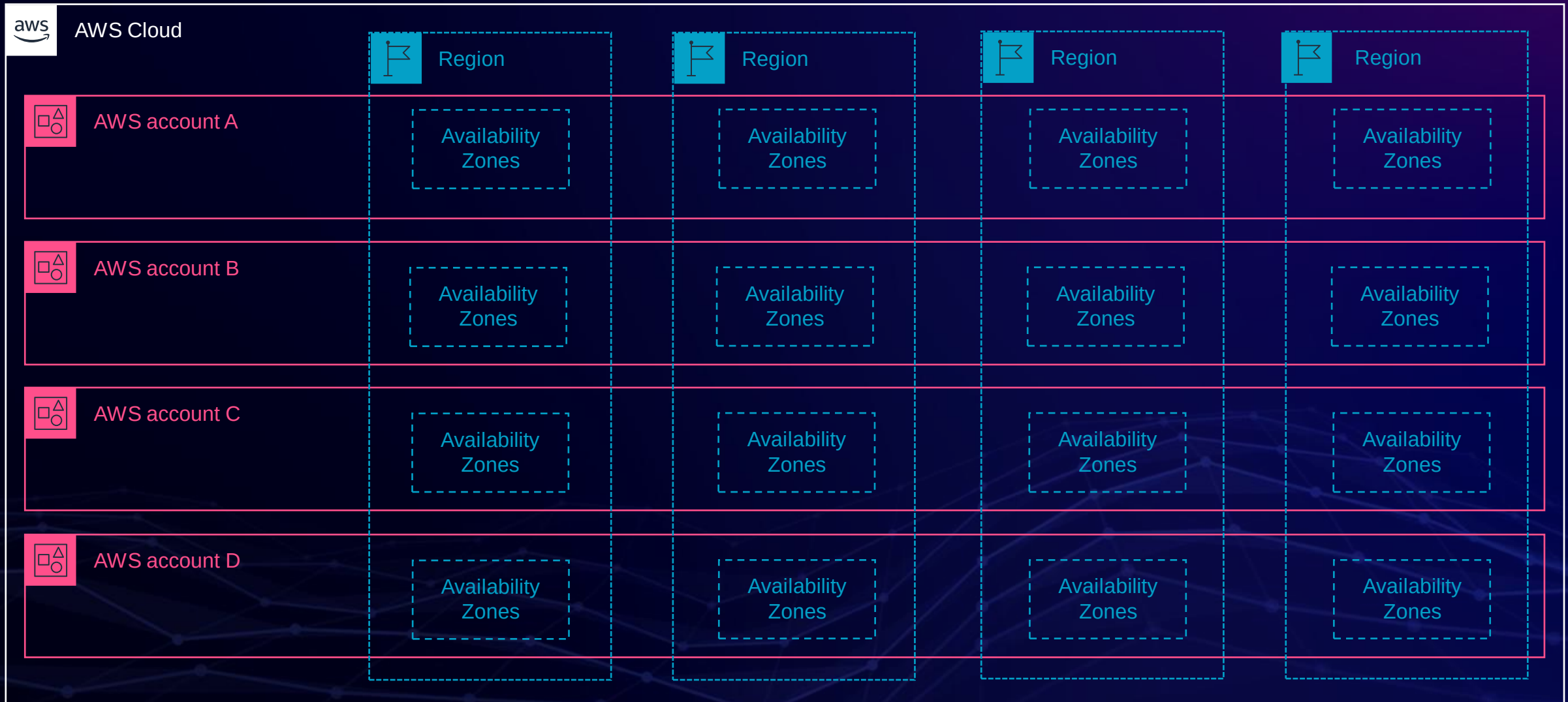
Principal Partner Solutions Architect
AWS



Agenda

- 01 Define multi-Region / multi-account
- 02 Lessons learnt from the field
- 03 Tools
- 04 Deployment examples
- 05 Ask me anything!

Accounts, Regions, Availability Zones



Why?

Why customers choose multi-account and multi-Region?

Data sovereignty, compliance, and regulations

Workload and data visibility

High availability and resiliency

Performance and latency

Business process and cost management

01 Define multi-Region / multi-account

02 Lessons learnt from the field

03 Tools

04 Deployment examples

05 Ask me anything!

Deployment use cases

Platform resources

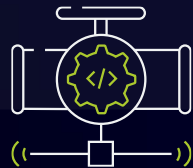
All accounts & all Regions

Common infrastructures

Prefers consistency

Large deployment targets

High-level abstraction



Workload resources

Select accounts & Regions

Specific infrastructures

Prefers order

Selected targets

High degree of controls



01 Define multi-Region / multi-account

02 Lessons learnt from the field

03 Tools

04 Deployment examples

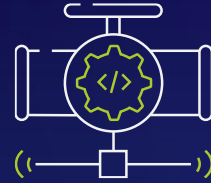
05 Ask me anything!

Deployment tools



IaC tools

AWS CDK
AWS CloudFormation
Terraform



CI/CD

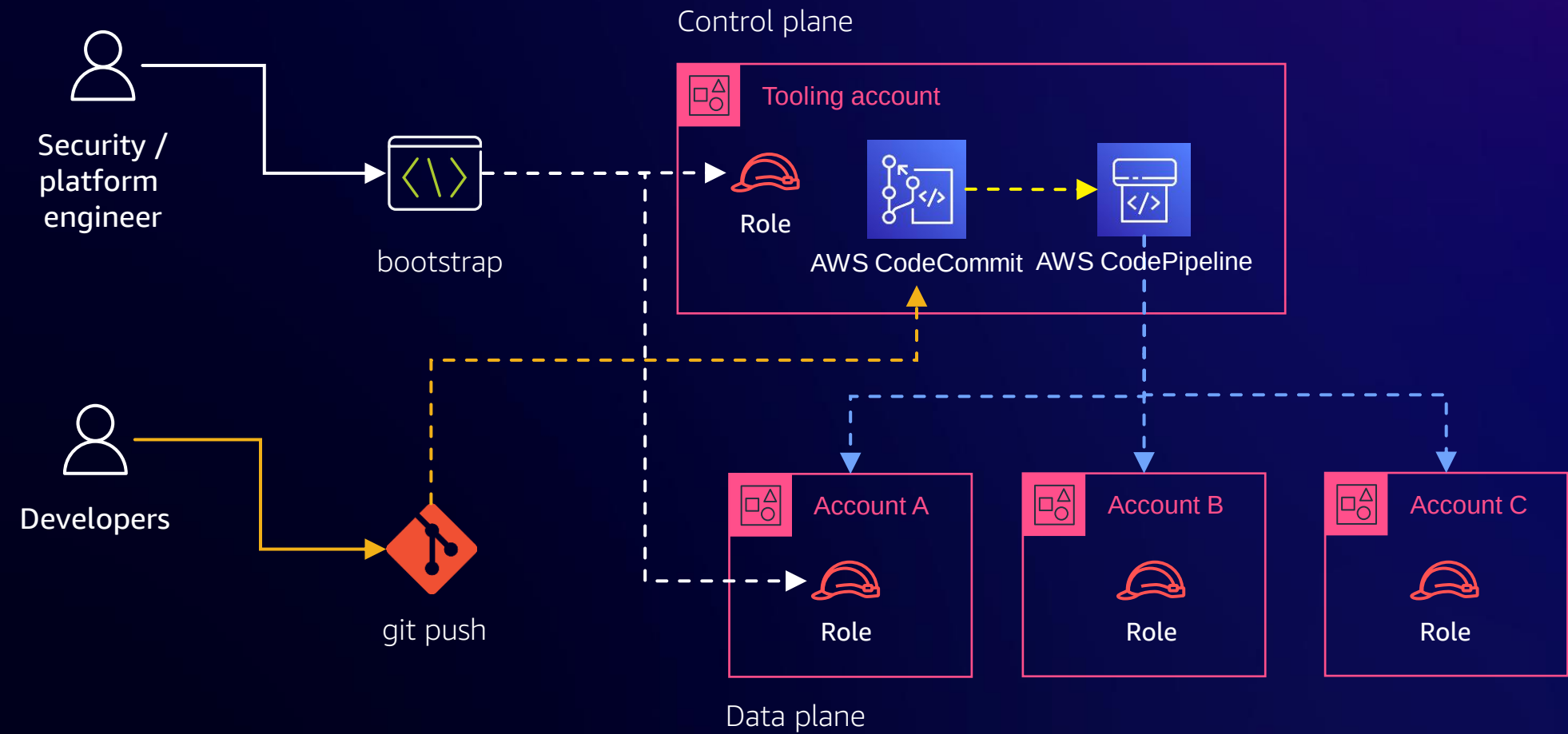
AWS CodePipeline
CDK Pipelines
Amazon CodeCatalyst



Other tools

Parameter Store, a capability
of AWS Systems Manager
AWS Secrets Manager
AWS Organizations
AWS Identity and Access
Manager (IAM)

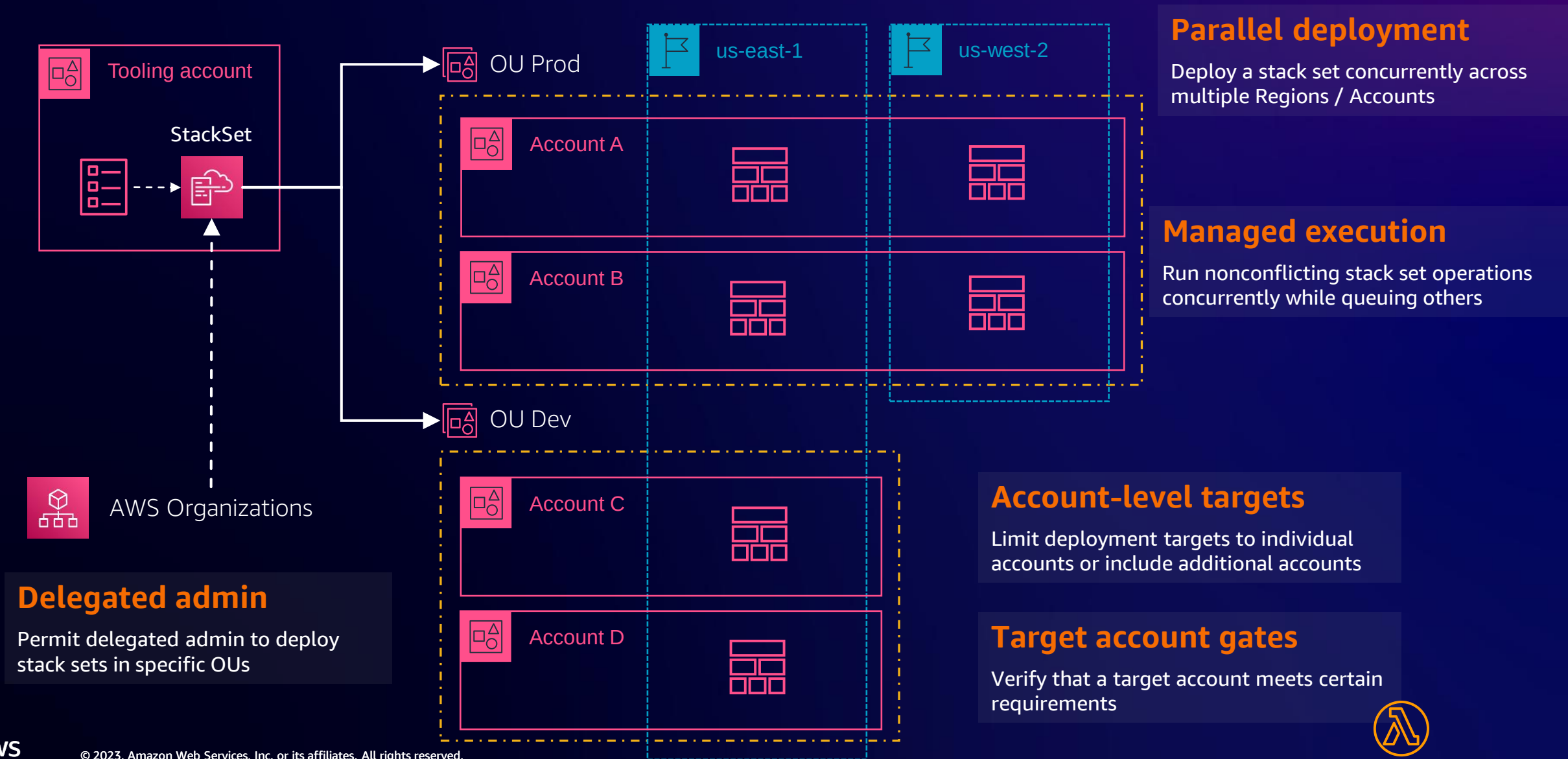
Deployment flow



IAM role example

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::CONTROL-PLANE:role/DEPLOYER"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

StackSet for platform resource deployment



Organizations example – StackSets

Set deployment options

Add stacks to stack set

☒ Deploy new stacks

☐ Import stacks to stack set

Deployment targets

StackSets deploys stack instances to all accounts in the target organization or organizational units (OUs). If you add a parent OU as a target, StackSets also adds any child OUs as targets. [Learn more](#)

☐ Deploy to organization

☒ Deploy to organizational units (OUs)

AWS OU ID

Enter OU

Remove

Add another OU

You can add 9 more OUs

Account filter type - optional

Set deployment targets to be specific individual accounts within OUs rather than entire OUs.

Select account filter type

Organizations example – StackSets continued

Specify regions

Choose the regions in which you want to deploy stacks. Stacks are deployed in these regions in the order that you specify. Note that during stack set operations, administrator and target accounts exchange metadata regarding the accounts themselves, as well as the stack set and stack set instances involved. [Learn more](#)

US West (Oregon)
us-west-2

^

v

Remove

^

v

Remove

Add all regionsRemove all regions

Deployment options

Maximum concurrent accounts - optional

Number of accounts per region to which you can deploy stacks at one time. The higher the number, the faster the operation

Number

1

Failure tolerance - optional

Number of account, per region, for which stacks can fail before CloudFormation stops the operation in that region. If the operation is stopped in one region, it does not continue in other regions. The lower the number the safer the operation.

Number

0

Region Concurrency

Choose to deploy StackSets into regions sequentially or in parallel.

☒ Sequential

Deploy StackSets operations into one region at a time, specified by the region deployment order.

☐ Parallel

Deploy StackSets operations into all specified regions in parallel.

Switching roles in build context

```
1 $ role_arn='arn:aws:iam::account_id:role/deploy-role'
2 $ role_session_name='deploy'
3 $ profile_name='deploy'
4
5 $ temp_role=$(aws sts assume-role \
6 $   --role-arn $role_arn \
7 $   --role-session-name $role_session_name)
8
9 $ export AWS_ACCESS_KEY_ID=$(echo $temp_role | jq -r .Credentials.AccessKeyId)
10 $ export AWS_SECRET_ACCESS_KEY=$(echo $temp_role | jq -r .Credentials.SecretAccessKey)
11 $ export AWS_SESSION_TOKEN=$(echo $temp_role | jq -r .Credentials.SessionToken)
12
13 $ aws configure set aws_access_key_id $AWS_ACCESS_KEY_ID --profile $profile_name
14 $ aws configure set aws_secret_access_key $AWS_SECRET_ACCESS_KEY --profile $profile_name
15 $ aws configure set aws_session_token $AWS_SESSION_TOKEN --profile $profile_name
```

01 Define multi-Region / multi-account

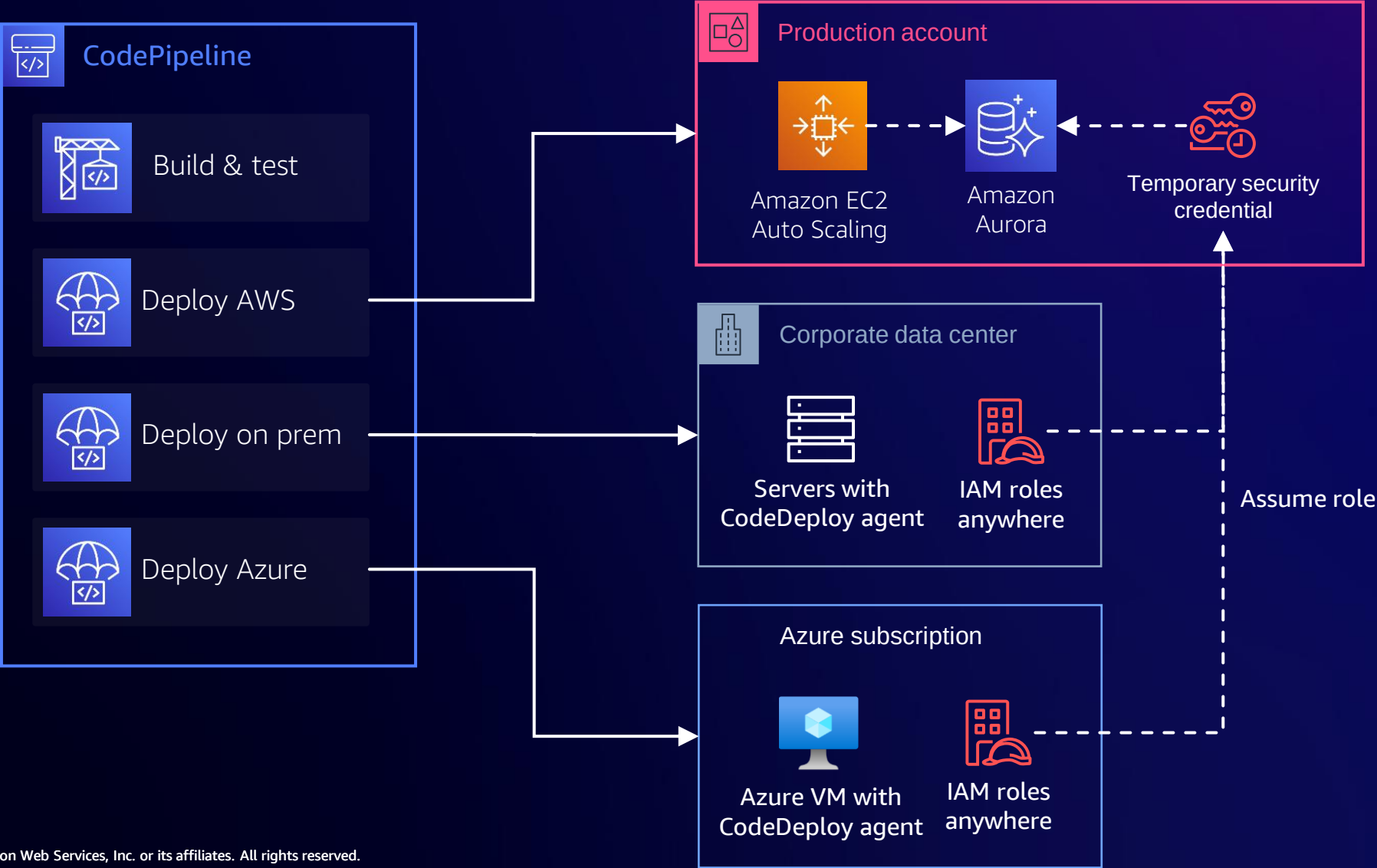
02 Lessons learnt from the field

03 Tools

04 Deployment examples

05 Ask me anything!

CodePipeline example



CDK Pipelines

```
export class PipelineStack extends Stack {
  constructor(scope: Construct, id: string, props?: StackProps) {
    super(scope, id, props);

    const sourceArtifact = new codepipeline.Artifact();
    const cloudAssemblyArtifact = new codepipeline.Artifact();

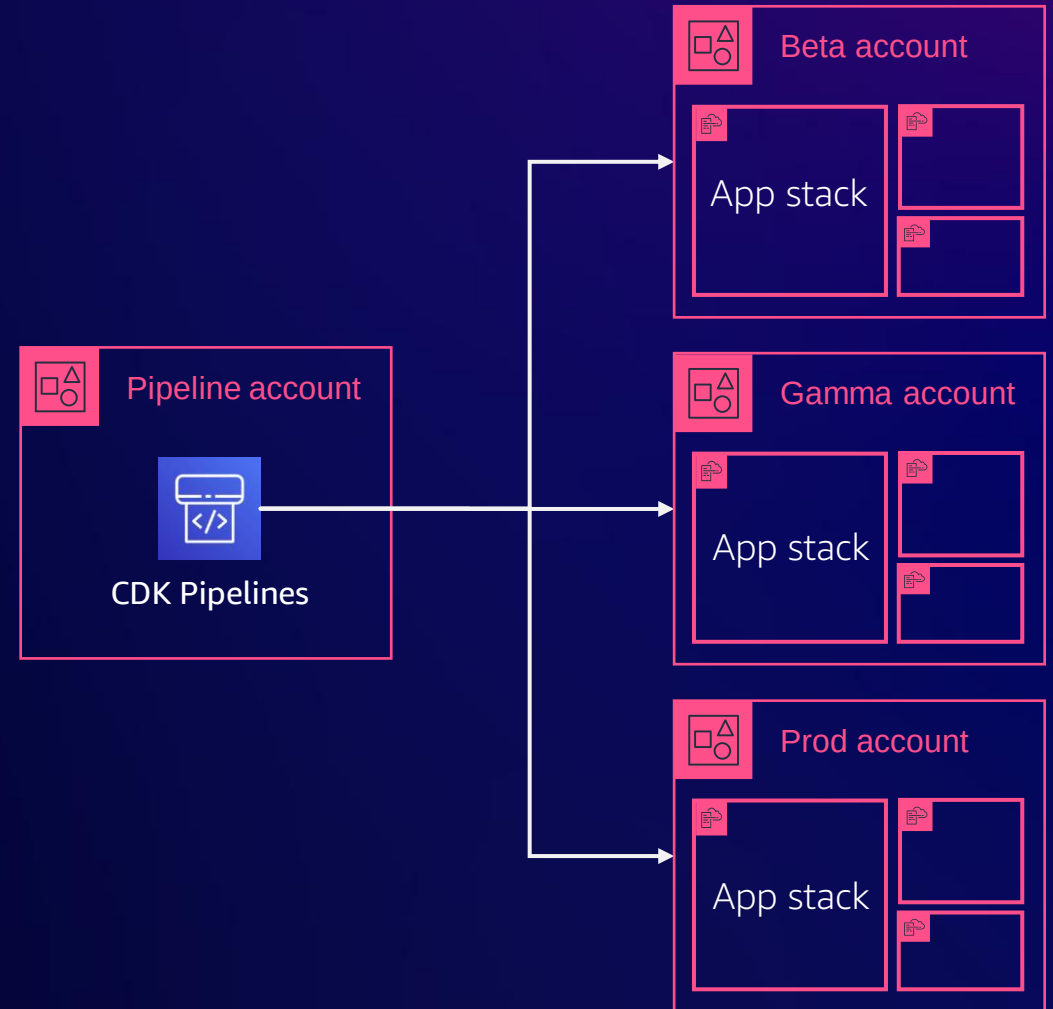
    const pipeline = new CdkPipeline(this, 'Pipeline', {
      cloudAssemblyArtifact,
      sourceAction: new cpa.GitHubSourceAction({ ...
    }},
    synthAction: SimpleSynthAction.standardNpmSynth({ ...
  });

  // Environment variables are *NOT* used here.

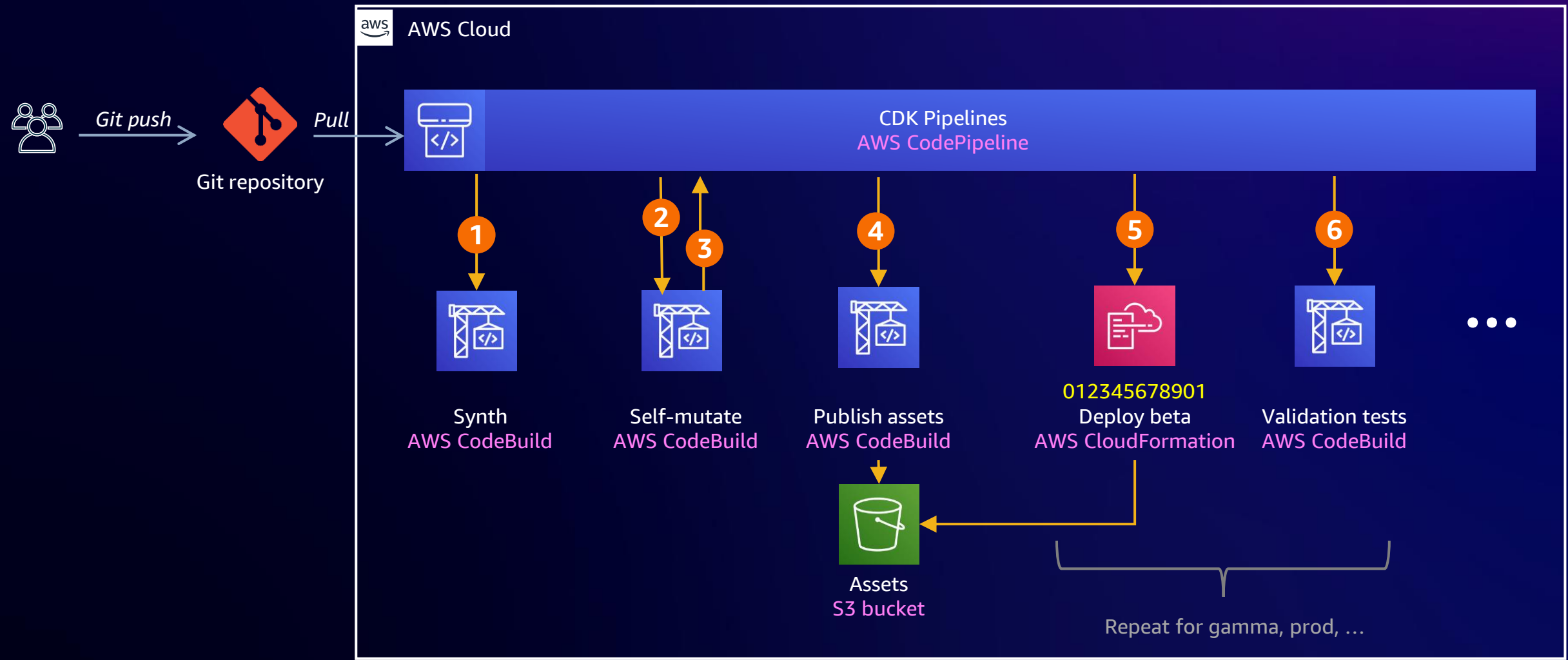
  pipeline.addApplicationStage(new DevoopsStage(this, 'Beta', {
    env: { account: '01234567891' }, // Beta Account
    domainName: 'beta-site.example.com'
  }));

  pipeline.addApplicationStage(new DevoopsStage(this, 'Gamma', {
    env: { account: '01234567892' }, // Gamma Account
    domainName: 'gamma-site.example.com'
  }));
}
```

✓ Model all production stages



CDK Pipelines workflow



CodeCatalyst

Action types

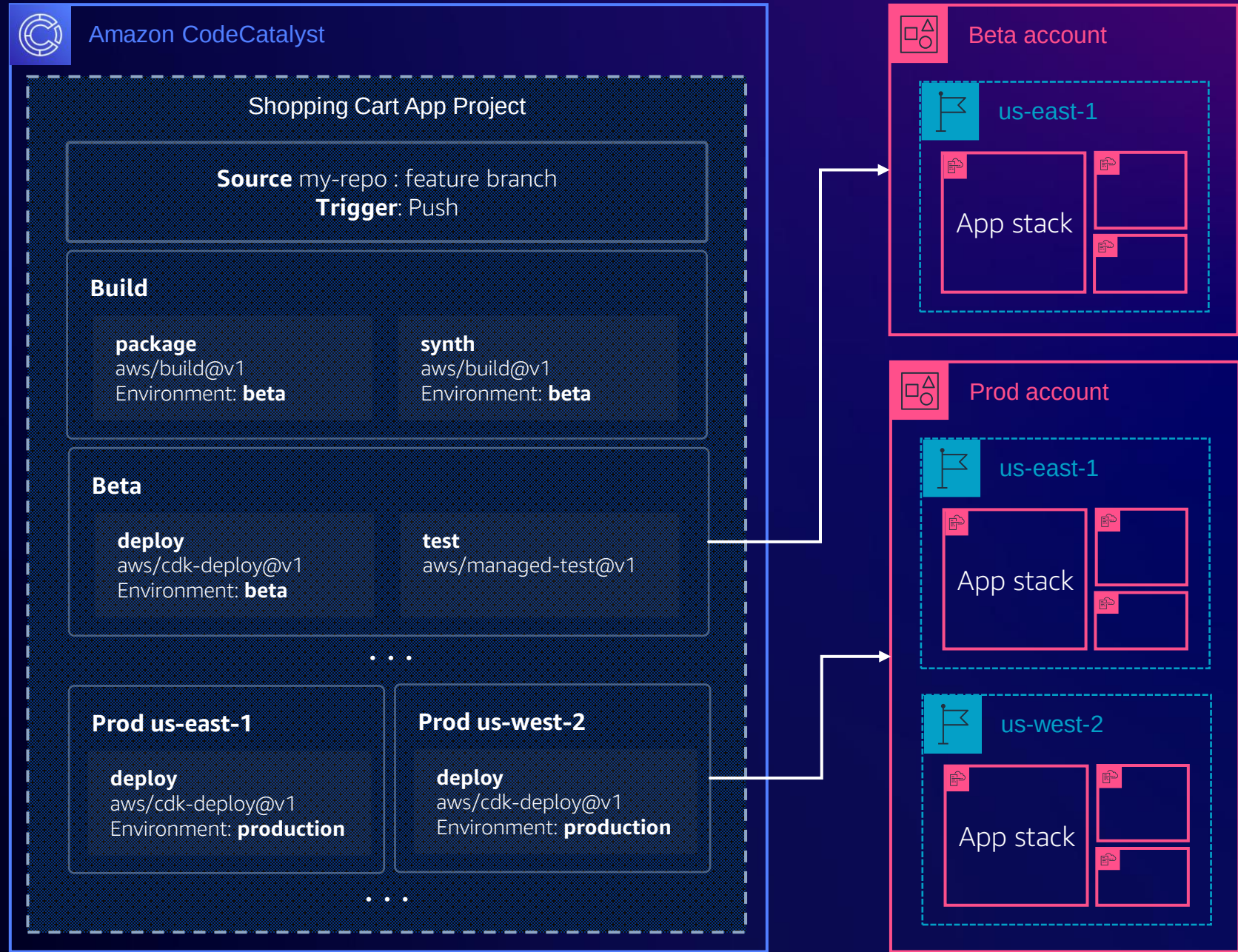
CodeCatalyst, CodeCatalyst Labs,
GitHub actions, third-party actions

Account connections

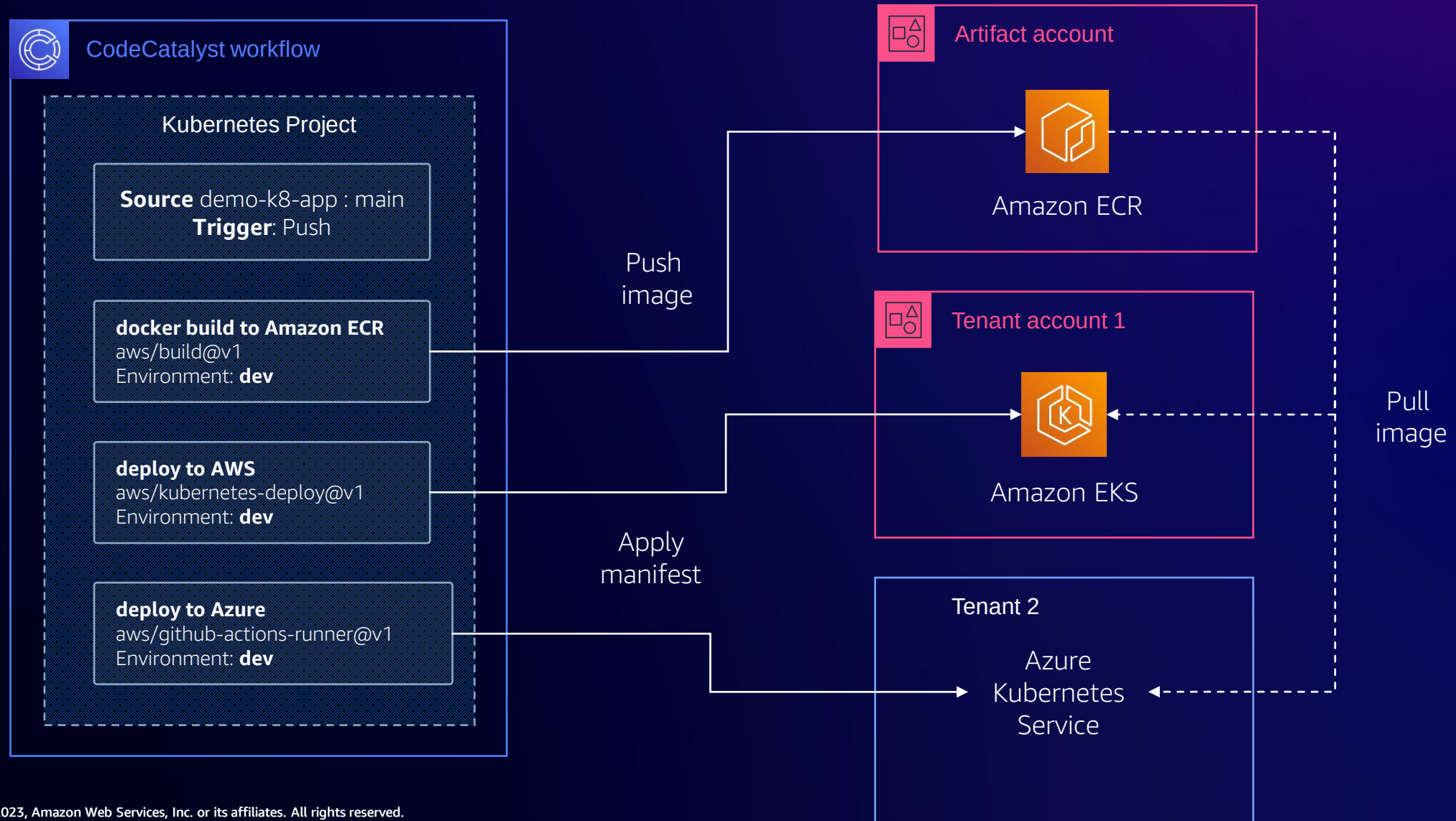
Connect to target AWS account using
trusted service principal

Deployment

CloudFormation, AWS CDK deploy,
CodeBuild, GitHub actions, Build
action, Amazon ECS, Amazon EKS, etc.



CodeCatalyst example



AWS application deployment methodology



Pipeline



Source &
build



Beta

Stack set :
1 account
1 Region



Gamma
One box

Stack set :
1 account
1 Region
Auto rollback

Bake time
Health checks
Metrics monitoring



Gamma

Stack set :
1 OU
4 Regions
Rolling deployment
Auto rollback

Bake time
Health checks
Metrics monitoring
Integration tests



Prod Wave 1
One box

Stack set :
1 account
1 Region
Rolling deployment
Auto rollback

Bake time
Health checks
Metrics monitoring

Alarm blocker
Time window blocker



Prod Wave 1

Stack set :
1 OU
1 Region
Rolling deployment
Auto rollback

Bake time
Health checks
Metrics monitoring
Integration tests

Alarm blocker
Time window blocker

...

canary test



Key takeaways

- Leverage AWS Organizations for platform resources
- Options for workload level resources
- Tooling support:
 - IaC: CloudFormation, Terraform, AWS CDK
 - CI/CD: CodePipeline, CDK Pipelines, CodeCatalyst, others...
 - Supporting: Parameter Store, AWS Secrets Manager, IAM roles

Ask me anything: Multi-account, multi-Region deployment

Thank you!



Please complete the session
survey in the mobile app

James Bland, PhD

[linkedin.com/in/jamesbland123](https://www.linkedin.com/in/jamesbland123)

Welly Siau

[linkedin.com/in/wellsiau](https://www.linkedin.com/in/wellsiau)

